

Perfume & Oudh Shop

Technical Architecture & Project Documentation

Version 1.0 | MVP Phase | May 2026

Document Type	Technical Architecture Document
Project Phase	Phase 1 — Minimum Viable Product (MVP)
Target Market	India & Middle East
Order Method	WhatsApp (no payment gateway in MVP)
Estimated Cost	~INR 800/year (domain only)

1. Project Overview

This document describes the complete technical architecture, technology stack, feature set, and phased development roadmap for a custom-coded e-commerce website for a Perfume and Oudh retail business. The website is designed to serve customers in India and the Middle East, starting with a lightweight MVP and scaling progressively.

1.1 Business Goals

- Establish an online presence for the perfume and Oudh brand
- Allow customers to browse products and place orders via WhatsApp
- Enable the admin (shop owner) to manage products and view orders
- Build a scalable foundation for future payment integration and user accounts
- Keep infrastructure cost near zero during the initial phase

1.2 MVP Scope — What Is Included

- Public product catalog (homepage, listing page, detail page)
- WhatsApp-based order flow — no payment gateway required
- Admin panel to add, edit, and delete products
- Admin view of incoming orders
- Mobile-responsive design suitable for phone shoppers
- Firebase Hosting with a custom domain

1.3 Out of Scope for MVP

- User registration and login
- Payment gateway (Razorpay / Stripe)
- Order tracking for customers
- Discount codes and coupons
- Product reviews and ratings
- Email marketing / newsletters
- Multi-language or multi-currency support

2. Technology Stack

The stack is chosen to minimise cost, reduce backend complexity, and remain manageable by a developer with basic coding knowledge. Firebase replaces the traditional server + database combination.

Layer	Technology	Purpose
Frontend	React.js	Build interactive UI — product pages, cart, order flow
Styling	Tailwind CSS	Utility-first CSS framework for fast, responsive styling
Database	Firebase Firestore	NoSQL cloud database — stores products, orders, admin data
Hosting	Firebase Hosting	Free, fast CDN hosting with SSL certificate included
Images	Cloudinary	Cloud image storage and optimised delivery for product photos
Orders	WhatsApp (wa.me)	Pre-filled WhatsApp message sent to admin when customer orders
Auth (Future)	Firebase Auth	User login via Email, Google, or Phone OTP — ready to enable
Payments (Future)	Razorpay + Node.js	Payment gateway requiring a small backend server when added
Code Editor	VS Code	Free code editor (Visual Studio Code by Microsoft)
Version Control	GitHub	Store and backup all source code, enable collaboration

2.1 Why Firebase Instead of Node.js or Django

A traditional web application requires three separate components: a backend server (Node.js or Django), a standalone database (PostgreSQL or MongoDB), and a hosting provider. Firebase consolidates all three into a single managed service operated by Google. This eliminates the need to manage servers, configure databases, handle security patches, or pay for a VPS.

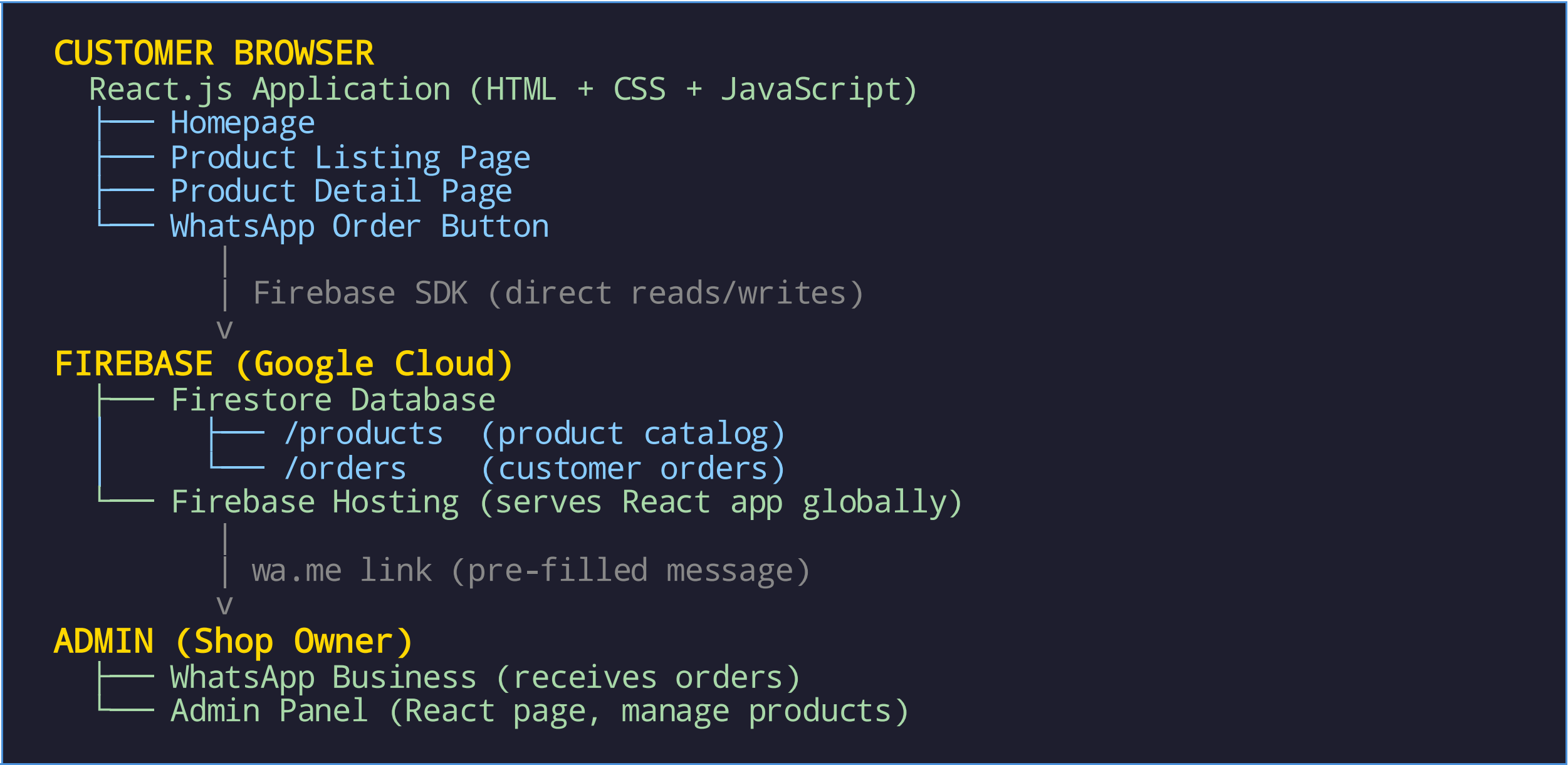
Factor	Firebase	Node.js	Django
Server Setup Required	No	Yes	Yes
Coding Expertise Needed	Basic	Intermediate	Advanced
Built-in Database	Yes (Firestore)	No	Yes (ORM)

Built-in Hosting	Yes (free)	No	No
Monthly Cost to Start	INR 0	~INR 500+	~INR 500+
Time to Set Up	1– 2 days	1– 2 weeks	2– 4 weeks

3. System Architecture

3.1 MVP Architecture Diagram

The diagram below describes the data flow from a customer browser through to the shop owner's WhatsApp.



3.2 Firestore Database Structure

Firestore is a NoSQL document database. Data is organised into collections (similar to tables) and documents (similar to rows). Below are the two primary collections used in the MVP.

Collection: products

```
products/  
  {productId}/  
    name      : string    - e.g. 'Rose Oudh Attar'  
    description : string    - full product description  
    price     : number     - price in INR  
    category  : string     - e.g. 'Attar', 'Oudh', 'Perfume'  
    imageUrl  : string     - Cloudinary image URL  
    stock     : number     - available quantity  
    featured  : boolean    - show on homepage?  
    createdAt : timestamp
```

Collection: orders

```
orders/  
  {orderId}/  
    customerName : string  
    customerPhone : string  
    items        : array    - [{productId, name, qty, price}]  
    totalAmount  : number
```

```
address      : string
status       : string    – 'pending' | 'confirmed' | 'shipped'
whatsappSent : boolean
createdAt    : timestamp
```

3.3 WhatsApp Order Flow

When a customer clicks the Order via WhatsApp button on a product page, the browser constructs a pre-filled message and opens WhatsApp. No payment is collected at this stage. The shop owner confirms the order manually and collects payment via UPI or cash on delivery.

1	Customer views a product and clicks 'Order via WhatsApp'
2	React constructs a wa.me URL with product name, price, and quantity
3	WhatsApp opens on customer's device with a pre-filled message to admin
4	Order is also saved to Firestore with status 'pending'
5	Admin receives the WhatsApp message, confirms order, collects payment via UPI
6	Admin updates order status in the admin panel to 'confirmed' or 'shipped'

4. Pages & Feature Breakdown

4.1 Customer-Facing Pages

Page	Components	Key Functionality
Homepage	Hero banner, featured products, brand tagline, WhatsApp CTA	First impression, drives product discovery
Product Listing	Product grid, category filter, search bar	Browse all products, filter by Attar / Oudh / Perfume
Product Detail	Image gallery, description, price, size options, Order button	WhatsApp order trigger, full product info
About Page	Brand story, sourcing, values	Build trust with customers
Contact Page	WhatsApp button, email, address	Customer support channel

4.2 Admin Panel Pages

Admin Page	Features
Admin Login	Simple password-protected page (Firebase Auth in future; hardcoded secret in MVP)
Product Manager	Add product (name, price, image, description, category, stock), edit, delete
Order Manager	View all orders sorted by date, update status (pending / confirmed / shipped)
Dashboard	Total orders count, pending orders, total products listed

5. Project Folder Structure

Below is the recommended folder layout for the React project. This structure separates concerns cleanly and scales well as features are added.

```
perfume-shop/
├── public/
│   └── index.html          – HTML entry point
├── src/
│   ├── components/        – Reusable UI components
│   │   ├── Navbar.jsx
│   │   ├── Footer.jsx
│   │   ├── ProductCard.jsx
│   │   └── WhatsAppButton.jsx
│   ├── pages/             – One file per page
│   │   ├── Home.jsx
│   │   ├── Products.jsx
│   │   ├── ProductDetail.jsx
│   │   ├── About.jsx
│   │   ├── Contact.jsx
│   │   └── admin/
│   │       ├── AdminLogin.jsx
│   │       ├── AdminDashboard.jsx
│   │       ├── ManageProducts.jsx
│   │       └── ManageOrders.jsx
│   ├── firebase/          – Firebase configuration
│   │   ├── config.js      – Firebase app initialisation
│   │   ├── products.js    – CRUD functions for products
│   │   └── orders.js      – CRUD functions for orders
│   ├── App.jsx            – Routes definition
│   └── index.js           – React entry point
├── .env                   – Firebase API keys (never commit to GitHub)
├── .gitignore
├── firebase.json          – Firebase Hosting config
└── package.json
```

6. Development Roadmap

Phase 1 — MVP (Current Plan)

Week	Tasks	Deliverable
Week 1	Install VS Code, Node.js, Firebase CLI. Create React app. Connect Firebase. Set up GitHub repo.	Project skeleton ready
Week 2	Build Navbar, Footer, Homepage with hero banner and featured products section.	Homepage live
Week 3	Build Product Listing page with grid and category filter. Build Product Detail page with WhatsApp order button.	Catalog pages live
Week 4	Build Admin panel — product CRUD, order viewer, status updater. Add Firestore security rules.	Admin panel working
Week 5	Buy domain. Deploy to Firebase Hosting. Test on mobile. Fix bugs. Go live.	Website live

Phase 2 — User Accounts

- Enable Firebase Authentication (Email/Password + Google Sign-In)
- Add Phone OTP login (ideal for Indian and Middle Eastern customers)
- Store order history per user in Firestore
- Show My Orders page after login
- Estimated timeline: 2– 3 weeks after MVP

Phase 3 — Payments

- Add a small Node.js backend server (3 API routes only)
- Integrate Razorpay for India payments
- Integrate Stripe for Middle East / international customers
- Payment confirmation saves to Firestore and triggers WhatsApp notification
- Host mini backend on Railway or Render (free tier available)
- Estimated timeline: 4– 6 weeks after Phase 2

7. Cost Breakdown

7.1 MVP Phase Costs

Service	Plan	Cost
Firebase Hosting	Spark (Free)	INR 0/month
Firebase Firestore	Spark (Free)	INR 0/month
Firebase Auth	Spark (Free)	INR 0/month
Cloudinary (images)	Free tier	INR 0/month
WhatsApp Business	Free app	INR 0/month
Domain Name	GoDaddy / Namecheap	~INR 800/year
TOTAL (Year 1)	All free tiers	~INR 800/year

7.2 Future Phase Additional Costs

Service	Cost When Added
Razorpay	2% per transaction — no monthly fee, no setup cost
Stripe (international)	2.9% + USD 0.30 per transaction
Railway / Render (mini backend)	Free tier available; ~USD 5/month if upgraded
Firebase Blaze Plan (if scaled)	Pay-as-you-go; free tier covers most small shops indefinitely

8. Security Considerations

8.1 Firestore Security Rules

Firestore allows you to define rules that control who can read or write data. In the MVP, the following rules should be applied:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Anyone can read products (public catalog)
    match /products/{productId} {
      allow read: if true;
      allow write: if false; // Only admin via Firebase Console
    }

    // Anyone can create an order; only admin can read/update
    match /orders/{orderId} {
      allow create: if true;
      allow read, update: if false; // Locked until Auth is added
    }
  }
}
```

8.2 Environment Variables

- Firebase API keys must be stored in a .env file, never hardcoded in source code
- The .env file must be listed in .gitignore so it is never pushed to GitHub
- Cloudinary upload presets should be unsigned only for MVP; switch to signed uploads in Phase 3

8.3 Admin Panel Protection

- In MVP, the admin panel route (/admin) can be protected by a simple hardcoded password check in React
- In Phase 2, replace this with Firebase Authentication — only whitelisted email can access admin
- Never expose admin routes in the public navigation menu

9. Developer Tools & Setup

9.1 Required Installations

Tool	Download URL	Purpose
Node.js (LTS)	nodejs.org	Run React and Firebase CLI
VS Code	code.visualstudio.com	Code editor
Git	git-scm.com	Version control
Firebase CLI	npm install -g firebase-tools	Deploy to Firebase Hosting
GitHub Account	github.com	Code backup and collaboration
Cloudinary Account	cloudinary.com	Product image hosting

9.2 Project Initialisation Commands

```
# 1. Create React project
npx create-react-app perfume-shop
cd perfume-shop

# 2. Install dependencies
npm install firebase react-router-dom
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p

# 3. Login to Firebase
firebase login

# 4. Initialise Firebase in project
firebase init
# Select: Firestore, Hosting
# Public directory: build
# Configure as single-page app: Yes

# 5. Deploy
npm run build
firebase deploy
```

10. Glossary

Term	Definition
React.js	A JavaScript library for building user interfaces using reusable components
Tailwind CSS	A utility-first CSS framework that applies styling via class names directly in HTML
Firebase	Google's Backend-as-a-Service (BaaS) platform providing database, hosting, and auth
Firestore	Firebase's NoSQL cloud database that stores data as documents inside collections
Cloudinary	A cloud service for uploading, storing, and delivering optimised images and videos
wa.me	A WhatsApp URL scheme that opens a chat with a phone number and a pre-filled message
MVP	Minimum Viable Product — the simplest version of a product that can be launched
NoSQL	A type of database that stores data in flexible JSON-like documents rather than tables
CDN	Content Delivery Network — servers distributed globally to serve files faster to users
SSL	Secure Sockets Layer — encrypts data between user browser and server (the https padlock)
API	Application Programming Interface — a way for two software systems to communicate
CRUD	Create, Read, Update, Delete — the four basic operations on any database

Document prepared for Perfume & Oudh Shop — MVP Technical Planning

Version 1.0 | May 2026 | Confidential